SubTrack: Architecture and Protocols

Flutter 3 + Dart 3 · Android · Software design pattern: **MVVM (Model–View–ViewModel) with the Repository pattern**

1. Chosen pattern: MVVM + Repository

Flutter rebuilds the UI from state, so MVVM fits it more naturally than classic MVC: there is no controller that pushes changes into views. Instead, a **ViewModel** holds the screen state and notifies listeners, and widgets rebuild automatically. We use the `provider` package with `ChangeNotifier` because it is the approach recommended in the Flutter docs and is simple enough for a semester project. A **Repository** sits between the ViewModels and the data sources, so the UI never knows whether a subscription came from Gmail or from manual entry.

VIEW · Flutter widgets (UI only, no logic)
WelcomeScreen · PermissionScreen · ScanScreen · HomeScreen (ListView / CategoryGridView) · CategoryDetailScreen · SubscriptionDetailScreen · EditSubscriptionScreen · SettingsScreen

▲ data / state changes ▼ user actions & method calls

VIEWMODEL · ChangeNotifier classes exposed with Provider
AuthViewModel · ScanViewModel · SubscriptionListViewModel (filters, view mode, totals) · SubscriptionDetailViewModel · SettingsViewModel

▲ data / state changes ▼ user actions & method calls

MODEL · entities + repository (single source of truth)
Subscription · Category · BillingCycle · SubscriptionStatus · **SubscriptionRepository** (merges Gmail results with manual edits)

▲ data / state changes ▼ user actions & method calls

SERVICES / DATA SOURCES · behind abstract interfaces
GoogleAuthService (google_sign_in) · GmailService (googleapis / http) · ReceiptParser + Categorizer · CatalogService (TMDB API, content overlap) · LocalDatabase (sqflite) · SecureStorage (flutter_secure_storage) · NotificationService (flutter_local_notifications)

2. How each part applies to SubTrack

Layer	Responsibility in this app	Example
View	Draws the screens from the Figma mockups, forwards taps to the ViewModel and rebuilds with `Consumer`/`context.watch`. Contains no business rules.	The "List | Categories" toggle calls `vm.setViewMode(ViewMode.categories)`.
ViewModel	Holds the UI state (loading, errors, filters, view mode), computes derived values (monthly total, spend per category, "renews soon") and calls the repository and services.	`SubscriptionListViewModel.monthlyTotal` converts yearly and weekly prices to a monthly value.
Model	Plain immutable Dart classes plus the repository, which reads and writes SQLite, merges new scan results without overwriting user edits, and exposes a `Stream` of subscriptions.	A user-changed category is stored with `categoryLocked = true`.

Services	Talk to the outside world: Google sign-in, the Gmail REST API, the email parser, local notifications. Each one implements an abstract interface, so it can be replaced with a fake in tests.	`FakeGmailService` returns sample emails for widget tests and demos.

Project structure

```
lib/
  main.dart                 // MultiProvider setup, routes, theme
  core/theme.dart           // colours + Inter type scale from the style guide
  models/                   // subscription.dart, category.dart, billing_cycle.dart
  data/repositories/        // subscription_repository.dart
  data/services/            // google_auth_service.dart, gmail_service.dart, receipt_parser.dart,
                            // local_database.dart, notification_service.dart
  viewmodels/               // auth_vm.dart, scan_vm.dart, subscription_list_vm.dart, ...
  views/screens/            // welcome, permission, scan, home, category, detail, edit, settings
  views/widgets/            // subscription_tile.dart, summary_card.dart, category_card.dart ...
```

Core model

```
class Subscription {
  final String id;              final String name;            // "Netflix"
  final double? price;          final String currency;        // null when free
  final BillingCycle cycle;     // weekly | monthly | yearly
  final String category;        final bool categoryLocked;    // user override
  final DateTime? nextRenewal;  final SubscriptionStatus status; // active | cancelled
  final String? senderEmail;    final List<String> sourceMessageIds;
  final int? remindDaysBefore;  final String? notes;
  bool get isFree => price == null || price == 0;
}
```

3. Key interfaces (Dart abstract classes)

```
abstract class AuthService {
  Future<GoogleAccount?> signIn();                // scopes: email, gmail.readonly
  Future<String> getAccessToken();                // refreshed automatically
  Future<void> signOut();                         // also revokes the token
}
abstract class EmailSource {                      // implemented by GmailService
  Stream<String> findBillingMessageIds({DateTime? since});
  Future<EmailMessage> getMessage(String id);
}
abstract class ReceiptParser {
  SubscriptionCandidate? parse(EmailMessage m);     // amount, currency, cycle, date, service
}
abstract class SubscriptionRepository {
  Stream<List<Subscription>> watchAll();
  Future<void> upsert(Subscription s);
  Future<void> mergeScanResults(List<SubscriptionCandidate> found);
  Future<void> delete(String id);   Future<void> clearAll();
}
abstract class ReminderScheduler {
  Future<void> schedule(Subscription s);   Future<void> cancel(String id);
}
abstract class CatalogSource {                    // implemented by TmdbCatalogService
  Future<int> countTitles(List<int> providerIds, String region);   // AND = on all of them
  Future<List<Title>> titlesOnAll(List<int> providerIds, String region, {int page = 1});
}
```

4. Protocols and external interfaces

Protocol / interface	Purpose in SubTrack	Package / endpoint
OAuth 2.0 + OpenID Connect	The user signs in with their own Google account and grants the read-only Gmail scope. The app never sees the password, only a short-lived access token.	`google_sign_in`; scopes `email`, `https://www.googleapis.com/auth/gmail.readonly`
HTTPS (TLS 1.2+)	Encrypts all traffic between the phone and Google. The token is sent as an `Authorization: Bearer` header.	`http` / `googleapis`

REST + JSON (Gmail API v1)	Search for billing emails, then download each one. Incremental sync fetches only new mail since the last scan.	`GET users/me/messages?q=…`
		`GET users/me/messages/{id}`
		`GET users/me/history?startHistoryId=…`

MIME + Base64url	Emails arrive as multipart MIME with base64url-encoded parts. The parser decodes the text/html parts and extracts the amount, currency, billing cycle and renewal date with regular expressions plus a table of known senders.	`dart:convert`, `html` package
SQL (SQLite)	Stores subscriptions on the device so the app works offline and opens instantly.	`sqflite`
Android Keystore	Stores tokens and the last `historyId` in encrypted storage.	`flutter_secure_storage`
Android notifications + alarms	Shows a renewal reminder N days before each charge, even if the app is closed.	`flutter_local_notifications`, `timezone`
Platform channels	The Flutter plugins above use MethodChannels to call native Android APIs (Google Play services, Keystore, AlarmManager).	Provided by the plugins
REST + JSON (TMDB API v3)	Content overlap: counts and lists films/series available on two streaming services in the user's country. Streaming availability data comes from JustWatch through TMDB.	`GET /3/discover/movie` and `/3/discover/tv` with `with_watch_providers`, `watch_region=TR`
Android Intents	"Open account page" opens the provider's website in the browser.	`url_launcher`

Example Gmail search query: `q = "newer_than:1y (category:purchases OR subject:(receipt OR invoice OR subscription OR renewal OR 'payment confirmation' OR 'free trial'))"`

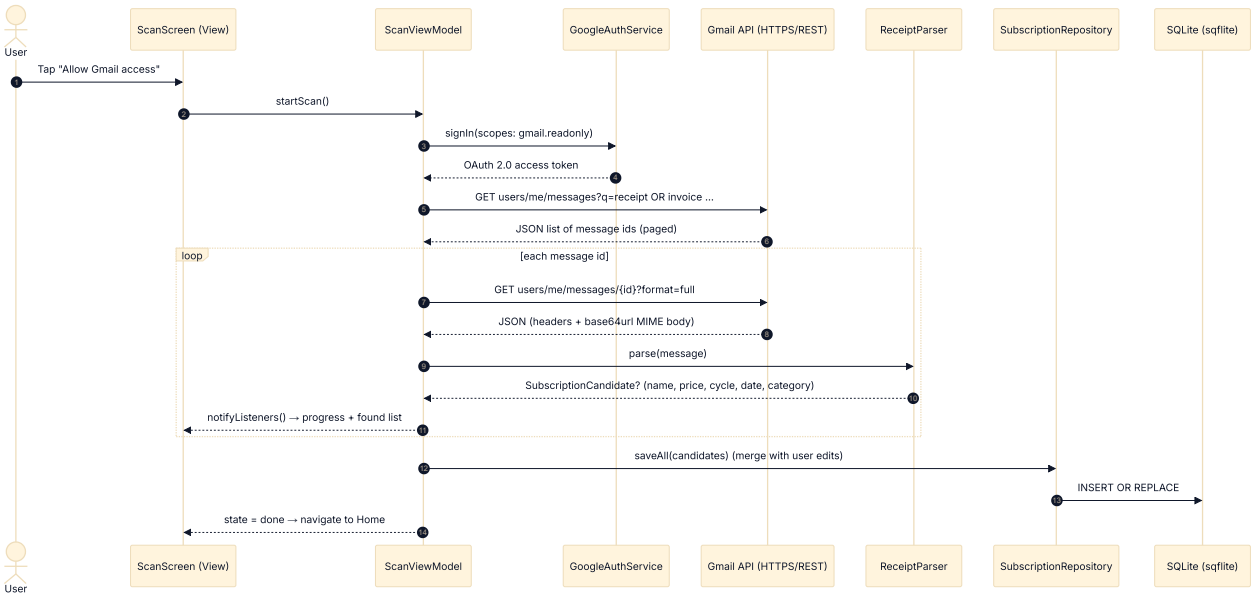
5. How content overlap works

Each detected streaming subscription is mapped to its TMDB provider id (Netflix = 8, Disney+ = 337, Apple TV+ = 350, MUBI = 11, Amazon Prime Video = 119). For a pair of services the app makes three cheap requests per media type and reads only the `total_results` field:

```
GET https://api.themoviedb.org/3/discover/movie?
watch_region=TR&with_watch_monetization_types=flatrate
    &with_watch_providers=8          → titles on Netflix            (A)
    &with_watch_providers=337        → titles on Disney+            (B)
    &with_watch_providers=8,337      → titles on BOTH (comma = AND)  (A∩B)
overlap % of Disney+ = (A∩B) / B      // shown as "11% of Disney+ is also on Netflix"
```

The same calls with `/discover/tv` cover series. Titles on both services are loaded page by page (20 per page) for the list, with posters from `image.tmdb.org`. Results are cached in SQLite for 7 days, so the screen opens instantly and stays within the free rate limit.

6. Sign-in and scan sequence



7. Security and privacy decisions

- **Least privilege:** only `gmail.readonly` is requested. The app cannot send, delete or modify email.
- **On-device processing:** email content is parsed on the phone and then discarded. Only the extracted subscription fields are stored. There is no own backend server.
- **Revocation:** "Sign out" and "Delete local data" revoke the token and wipe the database.
- **Course-project note:** `gmail.readonly` is a *restricted* Google scope. During development the OAuth consent screen stays in **Testing** mode with the team and the instructor added as test users (up to 100), which requires no Google security review. A public release would need Google's verification.
- **TMDB requests contain no personal data**, only provider ids and the region.

8. Cost: every service used is free

Service	Cost	Conditions
Google Sign-In + Gmail API	Free	No billing account or credit card is needed in Google Cloud. Usage is limited only by per-user rate quotas, which a personal inbox scan never reaches. Testing mode: up to 100 test users, and a test user has to sign in again every 7 days.
TMDB API	Free	Free API key for non-commercial projects; requires a TMDB account and the attribution "This product uses the TMDB API but is not

		endorsed or certified by TMDB", plus credit to JustWatch for streaming data.
Flutter packages	**Free**	All packages (provider, google_sign_in, googleapis, http, sqflite, flutter_secure_storage, flutter_local_notifications, url_launcher, google_fonts) are open-source.
Backend / hosting	**None**	Everything runs on the phone, so there is no server to pay for.